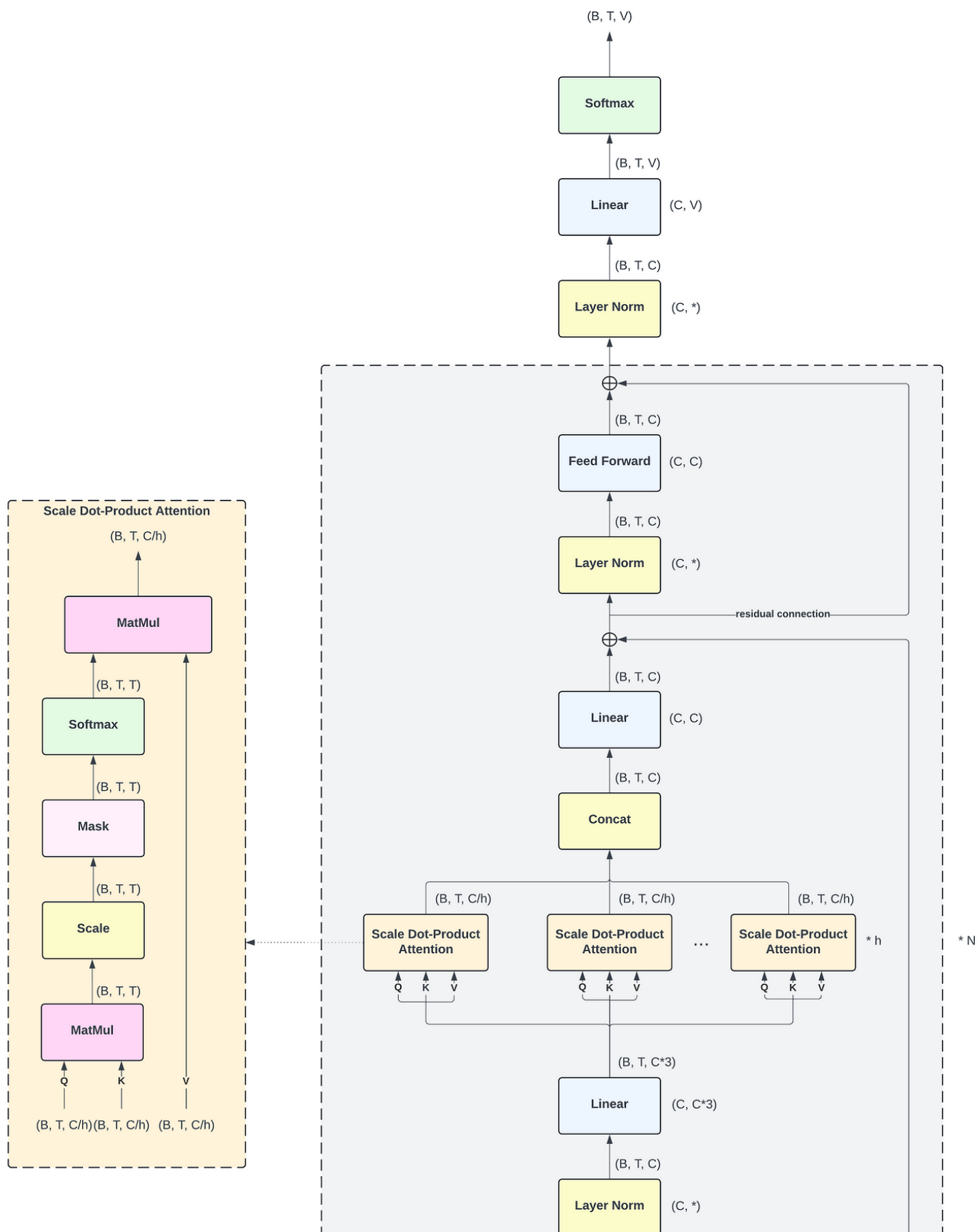
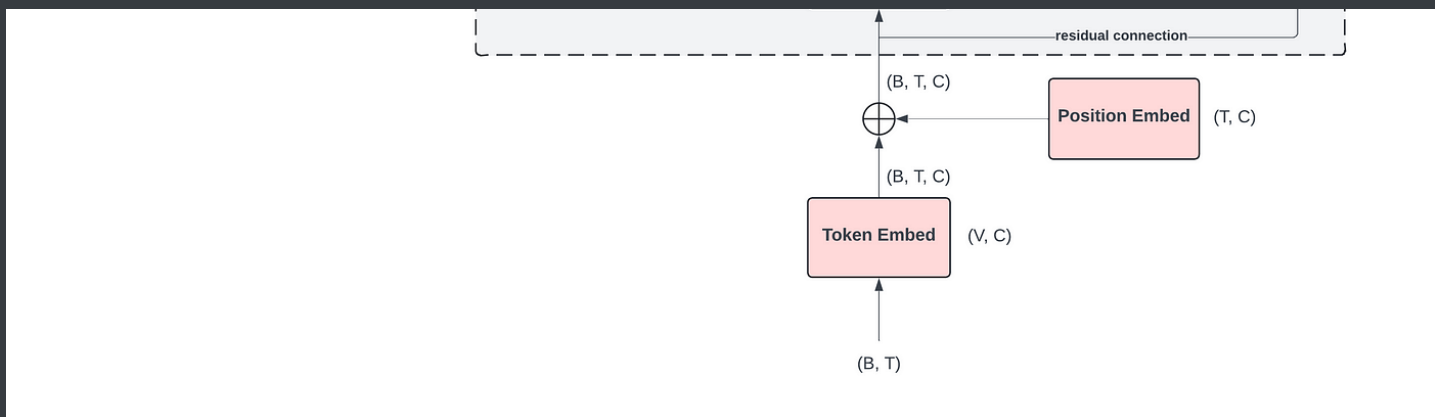


GPT2 手动实现





1、Single Head Attention 单头注意力结构实现（图中左侧黄色部分）

(1) 在 transformer 结构中，输入的数据 x 尺寸为：
 $\text{batch size} \times \text{sequence length} \times \text{channel}$ ，需要的数据结构是 $Q \times K \times V$ ，通过 `nn.Linear` 完成投影，适配下面要构建的多头注意力，并降低计算复杂度

- batch size：一次输入的样本量
- sequence length / time step：输入序列（单个样本）的 token 长度
- channel：每个 token 的特征维度
 - token 的特征维度牵扯出 embedding，此处举一个例子
 例如“我喜欢机器学习”，被分割后出现四个 token（我，喜欢，机器，学习），每个 token 映射到一个 ID（例如 我的 ID 为 234）。接下来进行 token embedding，每个 token ID 被映射为一个高维向量（`embedding_vector = [0.2, -0.5, 0.8, ..., 0.3]`，总共 768 个维度）
 - token embedding 的意义
 向量每个维度可以表示 token 的不同特征属性，并且词义相近的词的 token 在高维空间中距离相近，有助于模型更好地学习语意

(2) 进入图中粉色的 MatMul 和黄色的 Scale 部分，计算 QK 的相似度，根据 self-attention 公式 (1) 计算 QK^T ，并进行缩放 $/\sqrt{d_K}$

$$\text{Attention}(Q,K,V) = \text{softmax} \frac{QK^T}{\sqrt{d_K}} V \quad (1)$$

- 如果 Q_i 和 K_i 相似度高，对应位置的 QK^T 值较大；相似度低，对应位置的 QK^T 值较小
- 使用 $1/\sqrt{d_K}$ 缩放是为了防止梯度爆炸，其数学原理为：假设 Q, K 的数据随机生成，符合标准正态分布，计算后新矩阵的方差变为 $Var(QK^T) = d_K \times Var(Q) \times Var(K) = d_K$ ，即从放大了 d_K 倍，所以缩放来保持方差接近 1

(3) 然后进入图中淡粉色的 Mask 部分，对数据进行 Mask 掩码操作，防止模型偷看“未来”的 token

- Mask 是一个下三角矩阵，下三角部分为 1，表示模型可以看到的当前和历史 token；上三角部分为 0，表示模型不能提前查看的未来的 token

此处定义掩码时，有一个小 trick 可以提升计算效率：使用 register_buffer 将 attention mask 注册成一个 buffer，此 buffer 没有梯度，可以提高整体运行效率

```
self.register_buffer(
    "attention_mask",
    torch.tril(torch.ones(config.block_size, config.block_size))
)
```

然后根据掩码的 0-1，将对应位置的 attention score 替换为负无穷，使这些分数在 softmax 后被计算为 0

```
weight = weight.masked_fill(self.attention_mask[:seq_len, :seq_len] ==
0, float("-inf"))
```

(4) 然后过 softmax

(5) 最后和 V 计算最终的加权和，attention score (B, T, T) 表示每个 token 对所有 token 的注意力分布， V 表示每个 token 的信息表示，最终会得到 (B, T, C/h) 的注意力输出

2、Multi Head Attention 多头注意力结构实现

(1) 多头注意力结构对应图中右侧灰色区域中三个并列的 Scale dot-product attention，其本质上就是多个单头注意力结构并列组成（即通过 nn.ModuleList 多个并行组成）

```
self.heads = nn.ModuleList(  
    [SingleHeadAttention(config) for _ in range(config.n_head)]  
)
```

(2) 下一步拼接每个注意力头的输出，对应图中多头结构上方的黄色 Concat 部分

(3) 然后通过 Linear 层映射回到原始维度，对应图中 Concat 上方的 Linear 层。因为在上一步中 Concat 拼接后的结果维度为 $C * h$ ，所以要在这一步映射回原始的 C 维度；通俗理解成将模型多个头的信息融合，提高模型的表达能力

3、Feed forward 结构实现

Feed forward 部分对应图中灰色区域最上方的部分。前面构建的多头 transformer 结构主要负责从数据中提取关系，本身并不增加模型复杂度，因此此处加上 feed forward 来增加模型的非线性表示能力；并且 feed forward 是针对单个 token 的操作，不同 token 之间不会产生关联，因此可以并行运行

代码中拓展隐藏层与降回原始维度部分如下，此处的 4 是 transformer 中比较常用的前馈扩展比 4x

```
nn.Linear(config.hidden_dim, 4 * config.hidden_dim)
```

扩展之后再过一个 GELU 激活函数（transformer 中使用 GELU 比 RELU 多，因为相对而言 GELU 更加平滑，可以更好保留数据中的负数信息，规避 RELU 可能造成的稀疏性问题）

4、Transformer block 结构实现（整体堆叠重复 12 次）

上面定义的多头注意力和 feed forward 结构共同构成了完整的 transformer block，对应图中右侧整个灰色区域。需要注意的是，在定义数据传播过程时，不能遗漏右侧两个箭头代表的多头注意力 + 残差连接和 feed forward + 残差连接

```
x = x + self.att(self.ln1(x)) # 多头注意力 + 残差连接
x = x + self.ffn(self.ln2(x)) # FeedForward + 残差连接
```

5、剩余的 GPT 部分

(1) 首先构建 token embedding 和 position embedding，将二者的结果相加得到最终输入

```
self.token_embedding_table = nn.Embedding(config.vocab_size,
config.n_embd)
self.position_embedding_table = nn.Embedding(config.block_size,
config.n_embd)
```

(2) 归一化层，稳定模型，提升性能

```
self.ln_final = nn.LayerNorm(config.n_embd)
```

(3) 输出层

```
self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
```

(4) 权重共享，GPT 采用输入嵌入层和输出层参数共享，从而减少参数量，加快训练速度，提高训练稳定性

(5) 向前传播

6、生成函数

2、dataset 的构建

定义完 GPT 的结构后，开始构建 dataset

(1) 在 dataset 类的初始化中，通过 tiktoken 库调用 GPT2 的编码器，对数据进行编码，并且将特殊符号 <|endoftext|> 设为分隔符号

(2) 然后读取并处理 JSON 文件中的数据，此处设置最多读取 1000 行，防止数据读取过多导致 OOM 报错

(3) 通过上面调用的编码器对文本数据编码，并在每段文本末尾加上<|endoftext|>

```
full_encoded = []
for text in raw_data:
    encoded_text = self.enc.encode(text)
    full_encoded.extend(encoded_text + [self.eos_token])
```

(4) 然后根据编码好的数据生成训练样本，每个 block 的长度为 513（block size 前面设定为 512，因为要加上特殊符号所以长度加一）；如果长度不足，则用特殊符号填充至目标长度

```
for i in range(0, len(full_encoded), self.block_size):
    chuck = full_encoded[i: i + self.block_size + 1]
    if len(chuck) < self.block_size:
        chuck = chuck + [self.eos_token] * (self.block_size + 1 -
len(chuck))
    self.encoded_data.append(chuck)
```

(5) 此外，还需定义 len，getitem，encode，decode 方法